High-precision Bootstrapping for GCKKS

Shihe Ma, Jiaming Liu, Anyu Wang, and Xiaoyun Wang

Tsinghua University, Beijing, China
msh24@mails.tsinghua.edu.cn, liujm25@mails.tsinghua.edu.cn
anyuwang@tsinghua.edu.cn, xiaoyunwang@tsinghua.edu.cn

Abstract. Generalized CKKS (GCKKS) trades slot count for reduced modulus consumption in high-precision approximate arithmetic. However, designing a GCKKS bootstrapping algorithm with reduced modulus consumption throughout its computation remains a challanging problem. We construct high-precision bootstrapping for GCKKS using low-precision overflow extraction followed by iterative precision enhancement. Our main ingredient is a ciphertext-conversion framework for changing the decryption factor and slot count without requiring the factors to be units. A slot-aware analysis bounds unused-slot magnitudes proportionally to the corresponding evaluations of the decryption factor, which is essential to ensure the unused-slot values remain well-bounded after conversion. We combine precision doubling with ciphertext splitting and develop a joint evaluation procedure that produce two error-cleaned outputs at the cost of only one evaluation of the cleaning polynomial. We also give a useful-slot-preserving conversion from CKKS to GCKKS that simultanouesly resets unused slots through component-wise re-encoding. Underlying these constructions, we also improve the construction of the decryption factors by searching directly in the canonical embedding and obtain depth-free conjugation with non-unit factors. For target encoding precision p and $2N/k$ useful slots in a ring of dimension N, the precision-dependent stages of the bootstrapping uses $O(k)$ homomorphic operations and consumes $O((p/k)\log k)$ modulus bits.

Keywords: Fully homomorphic encryption · GCKKS · Ciphertext conversion · Bootstrapping.

1 Introduction

High-precision approximate homomorphic computation is constrained not only by the cost of each operation but also by the ciphertext modulus it consumes. In CKKS [7], increasing the scaling factor improves numerical precision but generally increases the modulus consumed by rescaling. Bootstrapping must recover an encryption of the message at a larger modulus while leaving sufficient modulus for subsequent computation. At high precision, its own modulus consumption can become a substantial part of the available budget [11].

Draft version, October 22, 2026.

Generalized CKKS (GCKKS) [20] changes this tradeoff by using fewer slots to support high-precision approximate arithmetic with smaller scaling factors. It incorporates a public polynomial F into decryption: a ciphertext $c = (b, a)$ is decrypted as $(b + as)F$ modulo the ciphertext modulus. The polynomial F has small evaluations on the useful slots, suppressing their errors, while larger errors are allowed in the unused slots. Its useful-slot precision can therefore match that of ordinary CKKS at a much larger scaling factor. The original GCKKS work develops this approach for leveled computation and outlines a bootstrapping construction in Section 1.4. This work develops that construction and the encoding, conversion, and noise-control techniques needed to realize it.

The leveled advantage does not automatically extend to bootstrapping. A straightforward reduction to ordinary high-precision CKKS by concentrating the useful-slot values into a subring of the same dimension can reintroduce large-scale intermediate computations and lose the desired modulus savings. Moreover, bootstrapping usually involves switching between different encoding precision and slot count, which introduces new challenges for managing both the magnitudes of useful- and unused-slot sizes in GCKKS.

We address these difficulties through an overflow-recovery approach. Building on the high-precision CKKS bootstrapping of Choe et al. [11], we extract the modulus-raising overflow at low precision and iteratively refine its encrypted digits. Rather than increasing the scaling factor as in ordinary CKKS, we change the GCKKS encoding and split ciphertexts as precision grows. This requires a conversion framework that simultaneously controls useful-slot precision, unused-slot magnitudes, and the rescaling divisors needed for further computation.

1.1 Our Contribution

Our contributions are summarized below.

- **Improved search for F.** We improve the construction of the GCKKS decryption factor F by formulating its search directly in the canonical embedding, extending its construction to a wider range of parameters to enable a more flexible control of useful- and unused-slot magnitudes. We further construct conjugate-invariant F using a lattice of half the original rank. This construction enables overhead-free homomorphic conjugation, whereas the original GCKKS construction requires an additional layer for non-unit F or an additional polynomial multiplication for unit F.
- **Generic GCKKS ciphertext conversion.** We develop a generic ciphertext conversion method between GCKKS ciphertexts with different encoding parameters. The core of the conversion is a slot-aware analysis to prevent the explosion of unused-slot size after the conversion, so that the noise-control conditions after conversion are independent of F.
- **High-precision GCKKS bootstrapping.** We develop a high-precision bootstrapping algorithm for GCKKS based on the low-precision overflow extraction and iterative precision improvement of Choe et al. [11]. For target

precision p and $2N/k$ useful slots in a ring of dimension N, the precision-dependent part of the bootstrapping cost is dominated by $O(k)$ homomorphic operations and $O(p \log k/k)$ bits of modulus consumption.

1.2 Technical Overview

Search for F. The original GCKKS construction searches for F by minimizing its useful-slot value and the size of its coefficients with different weights. However, since we are directly concerned with the canonical embedding of F, it is more natural to directly minimize its useful- and unused-slot values with different weights. We show that the resulting shortest-vector problem is equivalent to the original GCKKS search over its original weight range $w < 1$, and extends to the range of $w \geq 1$.

We further restrict F to the conjugate-invariant subring, reducing the search lattice to half its original rank. This enables homomorphic conjugation without additional computation or rescaling, even when F is not a unit. The restriction incurs a loss of only half a bit in the expected encoding quality.

Ciphertext Conversion with Slot-awareness. We give generic conversions between different GCKKS encoding parameters, including changes to both the encoding precision and the number of useful slots. The conversion works by reinterpreting the ciphertext under a new decryption factor. The main difficulty is that replacing F with F' multiplies the value at an embedding point ξ by $F'(\xi)/F(\xi)$. A uniform bound on unused-slot magnitudes can become excessively loose under this change. To solve this problem, the core observation is that the unused-slot magnitude at embedding point ξ is roughly proportional to $F(\xi)$. Then, let d_F be the polynomial recovered by F-decryption, we have the slotwise invariant bound K given by

$$\frac{|d_F(\xi)|}{|F(\xi)|} \leq K \qquad \text{for each unused embedding point } \xi.$$

For slots that remain unused, conversion preserves this bound of K since the old factor $|F(\xi)|$ cancels. We further handle slots that change between useful and unused status.

Our improved search for F allows to construct auxiliary polynomials with $w \geq 1$ to re-adjust the useful-slot magnitudes after ciphertext conversion. Moreover, the conjugate-invariant choice of F ensures that the complex phase of the useful-slot remains unchanged during the conversion, simplifying the conversion algorithms.

High-precision GCKKS Bootstrapping. Since both GBFV and GCKKS reduce the size of high-precision data by storing them in a larger dimension, an intuitive approach to bootstrap in very-high precision is to keep the useful data in this large-dimension format throughout bootstrapping. We combine this

idea with the low-precision overflow extraction and iterative cleaning of Choe et al. [11]. Using our ciphertext conversion methods, each iteration splits a ciphertext into two new ciphertexts with doubled the precision and halved useful-slot count. This design choice helps to keep the GCKKS scaling factors stable.

Given a GCKKS ciphertext of $c = (b, a) \in R_q^2$ with decryption factor F and decryption result $m \in R$, its bootstrapping consists of the following stages.

1. **Convert to standard CKKS.** Absorb F into the RLWE components to produce a standard CKKS ciphertext $(bF, aF) \in R_q^2$.
2. **Modulus raising.** Mod-raise the ciphertext above to get an encryption of $m + qI$ with $\|I\|_\infty$ bounded by a small integer K_I [3].
3. **Low-precision coefficents to slots.** Moves the coefficients of I into the slots of two CKKS ciphertexts via a CtS with small scale.
4. **Bit extraction.** Use functional bootstrapping to decompose the coefficients of I into bits.
5. **Iterative ciphertext conversion and precision doubling.** For each ciphertext, we split it into two ciphertexts with doubled k and squared effective CKKS scale. The error cleaning polynomials from [8] are used to reduce the error in the bits. This step is repeated until the value of k reaches that of the initial ciphertext c, producing $\lceil \log_2(2K_I + 1) \rceil \cdot k/2$ GCKKS ciphertexts.
6. **Recombine the bits.** The bits are recombined into the coefficients of I via a linear combination of ciphertexts, producing $k/2$ ciphertexts.
7. **Recovering the useful-slot values of I.** We use homomorphic linear transformation to recover the useful-slot values of qI inside a single ciphertext.
8. **Restoring the original F factor.** We convert the mod-raised ciphertext into a GCKKS ciphertext with decryption factor F. This step preserves the useful-slot values while resetting the unused-slot values to fresh modulus-switching noises.
9. **Subtraction.** We subtract the output ciphertext of Step 7 from that of Step 8. This recovers the useful-slot values of c and finishes the bootstrapping.

Steps 2,3,4 and 9 are identical to those from Choe et al.'s construction. In contrast, steps 5 to 8 are heavily modified to fit with GCKKS bootstrapping, which we detail below.

A direct implementation of step 5 would split a ciphertext first and evaluate the bit-cleaning polynomial $h_1(x) = 3x^2 - 2x^3$ [8,12] separately on both outputs. We instead develop a joint evaluation method that evaluates the powers of x required by h_1 before performing ciphertext splitting. Importantly, we do not restore the decryption factor to F after tensoring, but leave it as F^2. This doubles the relative precision bits of the relinearization and rescaling errors, thereby ensuring the precision of the useful-slot results. Finally, the two x powers are combined via plaintext masking to create the two output ciphertexts, followed by a final rescaling by Δ^3. See Figure 1 for a visualization of this procedure.

Step 7 can be viewed as evaluating a StC map whose inputs are stored in multiple ciphertexts, and outputting only the useful slots of StC. Interestingly,

the odd/even slot splitting in step 5 has a special interaction with this multi-ciphertext StC. Every layer of ciphertext splitting converts one FFT layer in the StC to a rotation-free linear combination of groups of four ciphertexts (see Figure 2 for an illustration). Therefore, the first $\log_2(k/4)$ layers in the StC and an extra real–imaginary combination layer combines the $k/2$ input ciphertexts into a single output via a weighted sum. The remaining StC layers are then evaluated on this ciphertext with GCKKS-compliant rotations. We remark that step 6 can be merged with step 7 as a larger linear transformation.

Restoring the useful slots of I and subtracting them from the mod-raised ciphertext is not sufficient for our bootstrapping, because the unused-slots still have a magnitude as large as q. Step 8 is designed to reset these unused slots with fresh rescaling noises before the subtraction, while keeping the useful slots in high precision. It extracts the useful-slot vector of each RLWE component, re-encodes it using the GCKKS encoder and a scale of one, and treats the output polynomials as the new RLWE components.

1.3 Parallel Works

Shaped arithmetic for small batches. Dumezy [13] gives a related scheme that trades unused slots for precision through shaped rounding, together with an implemented bootstrap, conversions to and from CKKS, and guard-slot control. His general construction requires the shaping polynomial to be invertible modulo the relevant moduli, and his concrete instantiations use units. His bootstrapping algorithm follows the StC-first design and concentrates the useful-slot data into the coefficients of a plaintext polynomial in a subring. This low-dimension intermediate representation reduces the number of data values to be processed by EvalMod, but also means that the data occupies modulus bits no less than its precision. Therefore, it is efficient for bootstrapping with moderate precision, but does not extend to bootstrapping with extremely high precision. Our construction instead targets a different regime of very-high precision bootstrapping, and does not require F to be invertible or a unit.

Focused CKKS. The initial version of Cheon and Lee's Focused CKKS [9] gives unit-based focused arithmetic, realizing similar tradeoff between slot count and modulus consumption. Their bootstrapping, however, requires the bottom modulus to hold the data values in its full precision, thereby having similar limitations as Dumezy's bootstrapping in high-precision scenarios.

Their revision [10, Section 6] additionally gives a discrete-CKKS-based bootstrap with focused cleaning, including a tree-like variant that splits ciphertexts as the precision increases. This revised construction is closely related to our bootstrapping framework.

The major difference between the two methods lies in the realization of the split-and-clean procedure. Their tree-like algorithm performs splitting before applying h_1 separately to the two children, while our joint evaluation shares the power computations and their relinearizations before the splitting. Each iteration in their method costs two levels in Δ, while ours costs three. Their

analysis keeps unused slots close to a discrete set during cleaning, whereas our conversion framework explicitly enforces the unused-slot noise control conditions on the newly created unused slots.

1.4 Relationship to the original GCKKS announcement.

The main ideas underlying our bootstrapping construction were outlined in Section 1.4 of the original GCKKS paper [21], including low-precision overflow extraction, iterative precision enhancement with ciphertext conversion. The present work develops these ideas into a complete construction with detailed algorithms and noise analysis.

2 Preliminaries

2.1 Notation

Let $\mathbb{Z}$, $\mathbb{R}$, and $\mathbb{C}$ denote the integers, real numbers, and complex numbers, respectively. Let $J = \sqrt{-1}$ and $\zeta_m = \exp(2\pi J/m)$ denote the primitive m-th root of unity. For a positive integer m, let $[m] = \{0, 1, \ldots, m-1\}$. Vectors and matrices are denoted by bold lowercase and uppercase letters, respectively. For a polynomial $f(X) = \sum_j f_j X^j$, the coefficient of X^j is denoted by f_j. The symbols $\|\cdot\|_2$ and $\|\cdot\|_\infty$ denote the Euclidean and maximum norms, respectively.

Throughout this paper, N is a power of two, and $\mathcal{R} = \mathbb{Z}[X]/(X^N+1)$ denotes the $2N$-th cyclotomic integer ring. For an integer modulus $q \geq 2$, let $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$. The canonical embedding of $f \in \mathcal{R}$ is defined as

$$\tau(f) = \left(f(\zeta_{2N}^{5^i})\right)_{i\in[N/2]} \in \mathbb{C}^{N/2}.$$

For a subset $S \subseteq [N/2]$, let τ_S denote the restriction of τ to the indices in S. The corresponding canonical norm is defined as $\|f\|_{\mathrm{can},S} = \|\tau_S(f)\|_\infty$, and $\|f\|_{\mathrm{can}} = \|\tau(f)\|_\infty$. For $j \in [2^r]$, let $\mathrm{rev}_r(j)$ denote the integer obtained by reversing the r-bit binary representation of j. The subscript is omitted when the bit length is clear.

2.2 Lattices

Let $\mathbf{B} \in \mathbb{R}^{m\times d}$ be a full-column-rank matrix with columns $\mathbf{b}_0, \ldots, \mathbf{b}_{d-1}$. The lattice generated by $\mathbf{B}$ is

$$\mathcal{L}(\mathbf{B}) = \{\mathbf{Bz} : \mathbf{z} \in \mathbb{Z}^d\},$$

with volume $\mathrm{vol}(\mathcal{L}(\mathbf{B})) = \sqrt{\det(\mathbf{B}^\mathsf{T}\mathbf{B})}$. The shortest vector problem (SVP) asks for a shortest nonzero lattice vector, while the closest vector problem (CVP) asks for a lattice vector minimizing the Euclidean distance to a given target.

BKZ reduction and the Geometric Series Assumption. BKZ reduction [5] applies lattice reduction to projected blocks of at most β dimensions, where β denotes the block size. Let $\mathbf{b}_0^*, \ldots, \mathbf{b}_{d-1}^*$ denote the Gram–Schmidt orthogonalization of $\mathbf{B}$. The quality of a BKZ-reduced basis is commonly estimated using the Geometric Series Assumption (GSA) [14]. Under this heuristic, the Gram–Schmidt lengths satisfy

$$\|\mathbf{b}_i^*\|_2 \approx \delta_\beta^{d-1-2i} \operatorname{vol}(\mathcal{L}(\mathbf{B}))^{1/d}, \qquad i \in [d],$$

where $\delta_\beta > 1$ denotes the root-Hermite factor associated with block size β.

2.3 The GCKKS Scheme

GCKKS [20] generalizes CKKS by trading the number of slots for reduced modulus consumption while preserving high-precision approximate arithmetic.

Useful slots and plaintext encoding. Let $k \geq 4$ be a power of two and set $\ell = 2N/k$. The ℓ useful slots occupy positions $ik/4$ in the full canonical embedding, with embedding points $\xi_i = \zeta_{2N}^{5^{ik/4}}$ for $i \in [\ell]$. We denote the useful- and unused-slot embeddings by τ_{useful} and τ_{unused}. The decryption factor $F \in \mathbb{Z}[X^\ell]/(X^N+1)$ has a common useful-slot value $c_m = F(\xi_i) \neq 0$, since $\xi_i^\ell = \zeta_k$.

Let $\texttt{iDFT}_{N,k}$ be the inverse canonical embedding in $\mathbb{C}[X]/(X^\ell - \zeta_k)$, and let $\texttt{Lift}_{N,k}$ extend its output to $\mathbb{R}[X]/(X^N+1)$ while setting unused slots to zero. For a message $\mathbf{v} \in \mathbb{C}^\ell$ and scale $\Delta > 0$, the encoded plaintext is MF, where

$$M' = \texttt{Lift}_{N,k}\big(\texttt{iDFT}_{N,k}(\Delta c_m^{-1}\mathbf{v})\big), \qquad M = \lfloor M' \rceil.$$

Rounding is coefficient-wise, and multiplication by F scales the useful-slot rounding errors by c_m and the unused-slot errors by the corresponding evaluations of F.

Encryption and decryption. Instead of encrypting MF directly, GCKKS encrypts M in the standard RLWE format and incorporates F into decryption. For a secret key s, we write $\texttt{RLWE}_{s,q}(M; e)$ for a ciphertext $\mathbf{c} = (b, a) = (-as + M + e, a) \in \mathcal{R}_q^2$, where a is uniform and e is the encryption noise. The augmented decryption is defined as

$$\texttt{Dec}_{s,F}(\mathbf{c}) = (b + as)F \pmod q.$$

Since $F(\xi_i) = c_m$ at every useful embedding point, the decrypted polynomial satisfies

$$\tau_{\text{useful}}\big(\texttt{Dec}_{s,F}(\mathbf{c})\big) = \Delta\mathbf{v} + c_m\tau_{\text{useful}}(M - M' + e).$$

Consequently, the useful-slot noise growth of GCKKS is comparable to that of standard CKKS with equivalent scale $\Delta/|c_m|$. Under standard CKKS decryption, the same ciphertext corresponds to the message $(|c_m|/c_m)\mathbf{v}$ at this equivalent scale. The unused-slot magnitudes must also be controlled to ensure correctness during homomorphic computation.

Standard CKKS [7] is the special case $k = 4$ and $F = 1$. In this case, all $N/2$ slots are useful, $c_m = 1$, the encoder reduces to $M = \lfloor \tau^{-1}(\Delta\mathbf{v}) \rceil$, and decryption computes $b + as \pmod q$.

Homomorphic operations. GCKKS uses the same RLWE computations as CKKS for `HomAdd`, `Rescale`, `HomPMult`, and `KeySwitch`, since these operations preserve the F-decryption format [20, Section 4.2]. Addition requires matching scaling factors and decryption factors, and plaintext multiplication uses the GCKKS-encoded polynomial MF. A useful-slot rotation by r positions uses a CKKS rotation by $rk/4$ positions, followed by the same key-switching procedure. The corresponding automorphism $\texttt{Rot}_{rk/4} : X \mapsto X^{5^{rk/4}}$ fixes F and preserves the useful-slot set.

Ciphertext multiplication and conjugation require additional operations because they change the decryption factor. In `HomCMult`, tensoring squares the decryption factor to F^2, so the resulting RLWE components are multiplied by F before the usual relinearization to restore F-decryption. In `HomConj`, the automorphism $\texttt{Conj} : X \mapsto X^{-1}$ changes the decryption factor to $\texttt{Conj}(F)$. Interpreting the conjugated ciphertext under F introduces an extra useful-slot phase $c_m/\overline{c_m}$, which the original GCKKS construction corrects by plaintext multiplication and rescaling when it is not one.

Both plaintext and ciphertext multiplication multiply the input scaling factors. Homomorphic linear transformations can be evaluated using the diagonal method [16], which expresses a matrix-vector product as a sum of rotated ciphertexts multiplied by plaintext vectors. The baby-step/giant-step (BSGS) method [17] reduces the number of rotations required for such transformations.

Unused-slot noise control. For $k > 4$, let $c_e = \|\tau_{\text{unused}}(F)\|_\infty$. We use the Gaussian noise model of GCKKS and denote the per-slot variances of RLWE encryption, coefficient-rounding, key-switching, and modulus-switching errors before multiplication by F by V_{enc}, V_{rnd}, V_{ks}, and V_{ms}, respectively. In particular, $V_{\text{enc}} = N\sigma_{\text{enc}}^2$ when the encryption error has coefficient variance σ_{enc}^2, $V_{\text{rnd}} = N/12$, and $V_{\text{ms}} = (1 + \|s\|_{\text{can}}^2)V_{\text{rnd}}$. A tail parameter $z > 0$ bounds each complex Gaussian error by z times its standard deviation with failure probability $\exp(-z^2)$ under this model. Let L bound the number of terms in a homomorphic linear transformation. The original GCKKS analysis [20, Lemmas 5 and 6] uses the encoding scale Δ as the rescaling divisor and gives the following propagation bounds.

Lemma 1 (Unused-slot noise propagation). *Suppose the unused-slot magnitudes of every input ciphertext are at most B. After a linear transformation with at most L terms or a ciphertext multiplication, each followed by rescaling by Δ, the output unused-slot magnitudes are bounded, respectively, by*

$$f_{\text{lin}}(B) = \frac{z\sqrt{LV_{\text{rnd}}}\,c_e}{\Delta}B + \frac{z^2L\sqrt{V_{\text{ks}}V_{\text{rnd}}}\,c_e^2}{\Delta} + z\sqrt{V_{\text{ms}}}\,c_e,$$
$$f_{\text{mult}}(B) = \frac{B^2}{\Delta} + zc_e\sqrt{\frac{V_{\text{ks}}}{\Delta^2} + V_{\text{ms}}}.$$

A freshly encrypted ciphertext has unused-slot magnitudes bounded by $B_{\text{fresh}} = zc_e\sqrt{V_{\text{enc}} + V_{\text{rnd}}}$.

Choose Δ so that f_{lin} has slope less than one and f_{mult} has two distinct nonnegative fixed points. Let B_{lin} be the fixed point of f_{lin} and B^{+}_{mult} the larger fixed point of f_{mult}. If

$$\max\{B_{\mathrm{fresh}}, B_{\mathrm{lin}}\} < B^{+}_{\mathrm{mult}},$$

then $[0, B^{+}_{\mathrm{mult}})$ is invariant under both propagation maps, so all unused-slot magnitudes remain below B^{+}_{mult} throughout a circuit composed of these operations. These bounds hold whenever the Gaussian tail estimates used in the circuit hold.

3 Improved GCKKS Encoding

3.1 Searching for F in Canonical Embedding

The multiplication-based encoder of GCKKS [20, Section 3.3] uses a polynomial F to adjust the magnitudes of useful- and unused-slot values. The original search constructs F from a short vector in a lattice. The coefficients of F, along with the real and imaginary parts of the useful-slot value, are embedded into two dimensions of the lattice with different weights. This means the search only indirectly minimizes the unused-slot values by minimizing the coefficients of F. We instead minimize both useful- and unused-slot values directly in the canonical embedding (with different weights). This gives a direct description of the search objective and extends the range of weight choices.

A weighted cyclotomic lattice. Let $k \geq 4$ be a power of two and set $n = k/2$. We search for a nonzero polynomial $F \in \mathbb{Z}[Y]/(Y^n + 1)$. Under the usual substitution $Y \mapsto X^{2N/k}$ into the full plaintext ring, $F(\zeta_{2n})$ becomes the common useful-slot value, and its other canonical evaluations give the unused-slot values. Here $\zeta_{2n} = e^{\pi\sqrt{-1}/n}$.

The canonical embedding over $\mathbb{Z}[Y]/(Y^n+1)$ corresponds to a complex evaluation matrix $\mathbf{B}' \in \mathbb{C}^{(n/2)\times n}$ defined by $\mathbf{B}'_{ij} = \zeta_{2n}^{5^i j}$. We consider its real representation

$$\mathbf{B} = \begin{pmatrix} \mathrm{Re}(\mathbf{B}') \\ \mathrm{Im}(\mathbf{B}') \end{pmatrix} \in \mathbb{R}^{n\times n}.$$

For a coefficient vector $\mathbf{z} \in \mathbb{Z}^n$, the vector $\mathbf{Bz}$ lists the real and imaginary parts of the canonical evaluations of $\sum_{j=0}^{n-1} z_j X^j$. Orthogonality of $\mathbf{B}'$ gives

$$\mathbf{B}^T\mathbf{B} = \mathrm{Re}\big((\mathbf{B}')^*\mathbf{B}'\big) = \frac{n}{2}\mathbf{I}_n.$$

Thus $\mathcal{L}(\mathbf{B})$ is a scaled orthogonal lattice of volume $(n/2)^{n/2}$.

To distinguish the useful slot from the others, choose $w > 0$ and define

$$\mathbf{D} = \mathrm{diag}(1, \underbrace{w, \ldots, w}_{n/2-1}, 1, \underbrace{w, \ldots, w}_{n/2-1}).$$

The search lattice is $\mathcal{L}(\mathbf{DB})$, whose volume is $w^{n-2}\,(n/2)^{n/2}$. We apply BKZ to this basis and take a short nonzero vector $\mathbf{b} = \mathbf{DBz}$. The corresponding polynomial is $F = \sum_{j=0}^{n-1} z_j X^j$. In particular, its coefficients are the integer coordinates of $\mathbf{b}$ in the weighted basis $\mathbf{DB}$, which can be recovered from the unimodular transformation recorded during reduction.

Quality of the resulting polynomial. By construction, the search objective is exactly

$$\|\mathbf{b}\|_2^2 = |F(\zeta_{2n})|^2 + w^2 \sum_{i=1}^{n/2-1} |F(\zeta_{2n}^{5^i})|^2.$$

Consequently, every vector returned by the search satisfies

$$|F(\zeta_{2n})| \le \|\mathbf{b}\|_2, \qquad |F(\zeta_{2n}^{5^i})| \le \frac{\|\mathbf{b}\|_2}{w}, \quad 1 \le i < n/2.$$

These bounds control the slot values directly, without passing through a bound on the coefficient norm.

Using the same Geometric Series Assumption as in the GCKKS analysis [20], the first vector of a BKZ-β reduced basis has estimated length

$$\|\mathbf{b}\|_2 \approx \delta_\beta^{n-1}\,\mathrm{vol}\big(\mathcal{L}(\mathbf{DB})\big)^{1/n} = \delta_\beta^{n-1}\sqrt{\frac{n}{2}}\,w^{1-2/n},$$

where δ_β is the root-Hermite factor associated with block size β. For fixed (k, β), this heuristic predicts useful- and unused-slot upper bounds of orders $O(w^{1-4/k})$ and $O(w^{-4/k})$, respectively. Thus small w gives the same precision–norm trade-off as the original GCKKS search.

Relation to the original GCKKS search. Let $\mathbf{e}_i$ denote the i-th standard basis vector and write

$$\mathbf{h}_0 = \mathbf{B}^T\mathbf{e}_0, \qquad \mathbf{h}_1 = \mathbf{B}^T\mathbf{e}_{n/2}.$$

These vectors contain the real and imaginary parts of $1, \zeta_{2n}, \ldots, \zeta_{2n}^{n-1}$. For every $\mathbf{z} \in \mathbb{R}^n$, we have

$$\|\mathbf{DBz}\|_2^2 = \frac{nw^2}{2}\|\mathbf{z}\|_2^2 + (1-w^2)\big(\langle \mathbf{h}_0, \mathbf{z}\rangle^2 + \langle \mathbf{h}_1, \mathbf{z}\rangle^2\big).$$

The original GCKKS search [20, Section 3.3] uses the column basis

$$\mathbf{B}_{k,w'} = \begin{pmatrix} \mathbf{h}_0^T \\ \mathbf{h}_1^T \\ w'\mathbf{I}_n \end{pmatrix}.$$

For $0 < w < 1$, setting

$$w' = w\sqrt{\frac{n}{2(1-w^2)}}$$

gives the exact identity

$$\|\mathbf{DBz}\|_2^2 = (1-w^2)\|\mathbf{B}_{k,w'}\mathbf{z}\|_2^2.$$

Hence the two shortest-vector problems are equivalent when $0 < w < 1$.

However, this equivalence does not extend to every positive w. When $w = 1$, the problem turns into minimizing $\|\mathbf{z}\|_2^2$; when $w > 1$, the term $\langle \mathbf{h}_0, \mathbf{z}\rangle^2 + \langle \mathbf{h}_1, \mathbf{z}\rangle^2$ has a negative weight, which cannot be incorporated into the norm minimization problem.

Another difference is that the new search supports using $w > 1$ to construct F with useful- and unused-slots magnitudes of $O(w^{1-k/4})$ and $O(w^{-k/4})$, which is not possible in the original GCKKS search. Indeed, as $w' \to \infty$, the $(w')^2 + k/4$ term in $\mathrm{vol}(\mathbf{B}_{k,w'})$ can no longer be approximated by $k/4$, and the original BKZ-based bounds on both types of evaluation remain $O(1)$ for fixed (k, β). This extra capability of the new search may be of independent interest.

3.2 Depth-free `HomConj` without a Unit F

The original GCKKS construction obtains depth-free homomorphic conjugation by choosing F to be a unit [20, Section 4.3]. This allows the decryption factor to be restored after conjugation by multiplying the RLWE components by $\mathtt{Conj}(F)F^{-1}$, whose canonical evaluations all have magnitude one. We relax this requirement by searching for a nonzero F satisfying

$$\mathtt{Conj}(F) = FY^i$$

for some integer i. The correction is then a multiplication by the monomial Y^i.

Restoring the decryption factor. Recall that a ciphertext $c = (b, a)$ decrypts as $\mathtt{Dec}_{s,F}(c) \equiv (b + as)F \pmod q$. After applying conjugation, multiply both RLWE components by Y^i to obtain

$$c' = \big(Y^i\mathtt{Conj}(b), Y^i\mathtt{Conj}(a)\big).$$

Under the conjugated secret key, this ciphertext satisfies

$$\begin{aligned}\mathtt{Dec}_{\mathtt{Conj}(s),F}(c') &\equiv Y^i\mathtt{Conj}(b+as)F \\ &= \mathtt{Conj}\big((b+as)F\big) \pmod q.\end{aligned}$$

The usual key switching from $\mathtt{Conj}(s)$ to s completes `HomConj`. Multiplication by Y^i, viewed as $X^{i\cdot 2N/k}$ in the full plaintext ring, is a signed permutation of the coefficients. It introduces no rounding error and preserves the magnitude of every canonical evaluation, including the existing noise. Thus the correction consumes no multiplicative depth or modulus.

The search for F can be collapsed into two cases. For every integer j,

$$\mathtt{Conj}(FY^j) = \mathtt{Conj}(F)Y^{-j} = (FY^j)Y^{i-2j}.$$

If i is even, choose $j = i/2$, and if i is odd, choose $j = (i-1)/2$. This reduces the search to two basic cases of

$$\mathtt{Conj}(F) = F \qquad \text{or} \qquad \mathtt{Conj}(F) = FY.$$

The case $\texttt{Conj}(F) = F$. All F in this case form the conjugate-invariant subring [18] of $\mathbb{Z}[Y]/(Y^n + 1)$ with basis

$$1, \qquad Y^j + Y^{-j}, \quad 1 \le j < n/2.$$

All canonical evaluations of these basis elements are real. Their evaluation matrix $\mathbf{B}_+ \in \mathbb{R}^{(n/2)\times(n/2)}$ is given by

$$(\mathbf{B}_+)_{i0} = 1, \qquad (\mathbf{B}_+)_{ij} = 2\cos\left(\frac{\pi 5^i j}{n}\right), \quad 0 \le i < n/2, \quad 1 \le j < n/2.$$

The coefficient vectors of the basis elements are orthogonal, with squared norms $1, 2, \ldots, 2$. Using $\mathbf{B}^T\mathbf{B} = (n/2)\mathbf{I}_n$ from the preceding construction gives

$$\mathbf{B}_+^T\mathbf{B}_+ = \mathrm{diag}(n/2, n, \ldots, n).$$

We apply the same weighted search to this smaller lattice. Using $\mathbf{D}_+ = \mathrm{diag}(1, w, \ldots, w) \in \mathbb{R}^{(n/2)\times(n/2)}$, we find a short vector $\mathbf{b} = \mathbf{D}_+\mathbf{B}_+\mathbf{z}$. The same useful- and unused-slot bounds follow from $\|\mathbf{b}\|_2$ and $\|\mathbf{b}\|_2/w$. The lattice now has rank $n/2$ and volume

$$\mathrm{vol}\big(\mathcal{L}(\mathbf{D}_+\mathbf{B}_+)\big) = w^{n/2-1}\sqrt{\frac{n}{2}\,n^{n/2-1}} = \frac{n^{n/4}}{\sqrt{2}}\,w^{n/2-1}.$$

Under the same BKZ heuristic, the first reduced vector has estimated length

$$\|\mathbf{b}\|_2 \approx \delta_\beta^{n/2-1}\sqrt{n}\,2^{-1/n}w^{1-2/n}.$$

For fixed (k, β), this predicts the same useful- and unused-slot upper-bound orders $O(w^{1-4/k})$ and $O(w^{-4/k})$ as the unrestricted search, with half the lattice dimension and a $\sqrt{2}$ times increase in $\|\mathbf{b}\|_2$. Depth-free conjugation is natural and the monomial correction is simply the identity in this case.

The case $\texttt{Conj}(F) = FY$. This condition defines a free $\mathbb{Z}$-module rather than a subring. Equivalently, F is fixed by the map $F \mapsto Y^{-1}\texttt{Conj}(F)$, which acts as an order-two signed permutation of the coefficient vector. Therefore, the module has basis

$$Y^j + Y^{-j-1}, \qquad 0 \le j < n/2.$$

Let $\theta_i = \pi 5^i/n$. At the i-th canonical embedding, the j-th basis element evaluates to

$$e^{j\theta_i\sqrt{-1}} + e^{-(j+1)\theta_i\sqrt{-1}} = 2e^{-\theta_i\sqrt{-1}/2}\cos\big((j+1/2)\theta_i\big).$$

I.e., multiplying each evaluation by $e^{\theta_i\sqrt{-1}/2}$ preserves its magnitude and makes it real. We can therefore use the real matrix

$$(\mathbf{B}_-)_{ij} = 2\cos\big((j+1/2)\theta_i\big), \qquad 0 \le i, j < n/2.$$

The basis coefficient vectors are orthogonal and all have squared norm two, so

$$\mathbf{B}_-^T\mathbf{B}_- = n\mathbf{I}_{n/2}, \qquad \mathrm{vol}\big(\mathcal{L}(\mathbf{D}_+\mathbf{B}_-)\big) = n^{n/4}w^{n/2-1}.$$

In conclusion, the two constrained lattices have the same rank, and their volume differ only by a factor of $\sqrt{2}$. We use the conjugate invariant case in the remaining of this paper because it requires no monomial correction in `HomConj`. Another reason is that reinterpreting a GCKKS ciphertext betwwen conjugate-invariant decryption factors does not change the complex phase of useful slots, which saves a lot of trouble for the ciphertext conversion in the next section.

4 Ciphertext Conversion in GCKKS

Ciphertext conversion changes the encoding parameters (k, w) in order to adjust the slot count and the encoding precision–norm trade-off, which will be useful for GCKKS bootstrapping. The conversion can be almost realized by replacing the decryption factor F with a factor F' for the target parameters, without changing the stored RLWE values. As an example, viewing a standard CKKS ciphertext as a GCKKS ciphertext is the special case with source decryption factor $F = 1$ and $k = 4$.

Ciphertext conversion introduces multiple challanges for noise control, including unused-slot bound control, change of encoding scale, and handling the conversion between useful and unused slots. The remaining of this section discuss these aspects in order.

Before beginning, we distinguish the GCKKS encoding scale Δ from the divisor $\rho \geq 1$ used in the next rescaling. The former only affects the useful-slot precision, while the latter is only used to control the unused-slot noise. The original GCKKS analysis takes these two quantities to be equal so that it can use the same Δ for every multiplication, but these two values can have different values after ciphertext conversion.

Without loss of generality, we assume the conversion preserves either k or w, since conversion from (k, w) to (k', w') can be realized by a k-preserving conversion to (k, w') followed by a w'-preserving one. We focus on k-preserving conversions first and leave the discussion of changing k to Section 4.3.

4.1 Slot-aware Noise Analysis for Unused Slots

The first obstacle in ciphertext conversion is controlling the unused-slot size after conversion. The original GCKKS analysis bounds them uniformly using $c_e = \max_{\xi \text{ unused}} |F(\xi)|$. At an individual embedding point ξ, however, replacing F with F' multiplies the decrypted value by $F'(\xi)/F(\xi)$[1]. This ratio cannot be effectively upper-bounded because it may has $|F(\xi)| \ll c_e$, leading to oversized unused slot magnitudes after conversion.

[1] $F(\xi)$ is nonzero since F is a nonzero element in $\mathbb{Q}(\zeta_k)$.

To solve this issue, the core observation is that the unused-slot propagation formulas remain valid with c_e replaced by $|F(\xi)|$. This is because polynomial operations act slotwise and thereby do not mix different slots of F. Also, a valid GCKKS rotation fixes F, so all slots mixed by such rotations have the same value of $F(\xi)$. Conjugation also preserves F under our choice of a conjugate-invariant F. We will later show that the per-unused-slot noise size is linear in $|F(\xi)|$, so that the factor $|F(\xi)|$ cancels during conversion.

Applying the analysis of Lemma 1 slotwise gives Lemma 2, where a_ξ denotes $|F(\xi)|$.

Lemma 2. *With the same noise parameters, the propagation maps at unused embedding point ξ are*

$$\begin{aligned} f_{\mathrm{lin},\xi}(B_\xi) &= \frac{z\sqrt{LV_{\mathrm{rnd}}}\,a_\xi}{\rho}B_\xi + \frac{z^2L\sqrt{V_{\mathrm{ks}}V_{\mathrm{rnd}}}\,a_\xi^2}{\rho} + z\sqrt{V_{\mathrm{ms}}}\,a_\xi, \\ f_{\mathrm{mult},\xi}(B_\xi) &= \frac{B_\xi^2}{\rho} + za_\xi\sqrt{\frac{V_{\mathrm{ks}}}{\rho^2} + V_{\mathrm{ms}}}, \\ B_{\mathrm{fresh},\xi} &= za_\xi\sqrt{V_{\mathrm{enc}} + V_{\mathrm{rnd}}}. \end{aligned}$$

Here B_ξ bounds the inputs at ξ and any slots rotated into that position.

A common normalized noise bound. According to the choice of minimum ρ in [20, Theorem 2], with other parameters fixed, the noise control model only requires $\rho/a_\xi \geq \rho/c_e$ to be lower-bounded by a constant. We therefore normalize the unused-slot noise sizes with respect to a_ξ.

If $\rho/a_\xi \geq r$ for some common $r > 0$ for all unused embedding points ξ, dividing the unused-slot magnitude by a_ξ gives the normalized bound $B := B_\xi/a_\xi$ and $B_{\mathrm{fresh}} := z\sqrt{V_{\mathrm{enc}} + V_{\mathrm{rnd}}}$ with propagation maps

$$\begin{aligned} \frac{f_{\mathrm{lin},\xi}(a_\xi B)}{a_\xi} &\leq g_{\mathrm{lin},r} := \frac{z\sqrt{LV_{\mathrm{rnd}}}}{r}B + \frac{z^2L\sqrt{V_{\mathrm{ks}}V_{\mathrm{rnd}}}}{r} + z\sqrt{V_{\mathrm{ms}}}, \\ \frac{f_{\mathrm{mult},\xi}(a_\xi B)}{a_\xi} &\leq g_{\mathrm{mult},r} := \frac{B^2}{r} + z\sqrt{V_{\mathrm{ks}} + V_{\mathrm{ms}}}. \end{aligned}$$

The second inequality uses $\rho \geq 1$ to obtain a ρ-independent bound. Note that these two maps do not explicitly depend on ρ or a_ξ.

For $r > z\sqrt{LV_{\mathrm{rnd}}}$ and $r > 4z\sqrt{V_{\mathrm{ks}} + V_{\mathrm{ms}}}$, their fixed points are

$$\begin{aligned} B_{\mathrm{lin}}(r) &= \frac{z^2L\sqrt{V_{\mathrm{ks}}V_{\mathrm{rnd}}}/r + z\sqrt{V_{\mathrm{ms}}}}{1 - z\sqrt{LV_{\mathrm{rnd}}}/r}, \\ B^{\pm}_{\mathrm{mult}}(r) &= \frac{r}{2}\left(1 \pm \sqrt{1 - \frac{4z\sqrt{V_{\mathrm{ks}} + V_{\mathrm{ms}}}}{r}}\right) < r. \end{aligned}$$

Choose r large enough that

$$K := \max\{B_{\text{fresh}}, B_{\text{lin}}(r)\} < B^{+}_{\text{mult}}(r),$$

where a possible choice[2] is

$$r = z \max\{2\sqrt{L}\sqrt[4]{V_{\text{ks}}V_{\text{rnd}}}, 4\sqrt{V_{\text{ks}} + V_{\text{ms}}}, \sqrt{V_{\text{enc}} + V_{\text{rnd}}}\}.$$

The interval $[0, K]$ is invariant under both normalized maps. Indeed, $K \geq B_{\text{lin}}(r)$ gives $g_{\text{lin},r}(K) \leq K$, and $K < B^{+}_{\text{mult}}(r)$ gives $g_{\text{mult},r}(K) \leq K$. Both maps are increasing on the nonnegative real axis. Induction over the circuit therefore gives the slotwise bound

$$|\mathtt{Dec}_{s,F}(c)(\xi)| \leq K|F(\xi)|, \qquad \xi \text{ unused}, \tag{1}$$

provided every rescaling divisor satisfies $\rho \geq 1$ and $\rho/|F(\xi)| \geq r$.

Unused-slot noise after *k*-preserving conversion. Write the underlying noisy RLWE message as u, so that GCKKS decryption gives uF. Replacing F with F' leaves the stored pair (b, a) unchanged and changes the intended decryption to uF'. At every embedding point,

$$(uF')(\xi) = (uF)(\xi)\frac{F'(\xi)}{F(\xi)}.$$

For a slot that is unused in both parameter sets, (1) immediately gives

$$|(uF')(\xi)| \leq K|F'(\xi)|.$$

Hence the normalized unused-slot bound is unchanged by conversion. Choosing the new rescaling divisor ρ' to satisfy

$$\rho' \geq \max\{1, rc'_e\}, \text{ where } c'_e = \max_{\substack{\xi \\ \text{unused}}} |F'(\xi)|$$

preserves the same invariant interval under the target operations. This proves stability for ciphertexts already produced by a circuit, in addition to freshly encrypted ciphertexts.

4.2 Choosing the Encoding Scale and Rescaling Divisor

We have focused on controlling the size of unused-slot values, and now we include the useful-slot values into our analysis.

Let c_m and c'_m be the common useful-slot values of F and F'. Since the factors are conjugate-invariant, we may assume both values are real and positive. The conversion multiplies each useful slot by c'_m/c_m, leading to

$$\Delta' = \Delta\frac{c'_m}{c_m}.$$

[2] The first terms implies $r \geq 2z\sqrt{LV_{\text{rnd}}}$ since $V_{\text{ks}} > V_{\text{rnd}}$.

If $\Delta' \geq \max\{1, rc'_e\}$, we can set $\rho' = \Delta'$ to stabilize both the encoding scaling factor and unused-slot sizes during homomorphic multiplications. However, if Δ' is smaller, any valid choice of ρ' is too large to stabilize the encoding scale. We therefore need to increase the post-conversion encoding scale without increasing the unused-slot sizes.

A striaghtforward way to achieve this is through one HomPMult followed by rescaling. Alternatively, we give a method that does not consume multiplicative layers. We could search for an auxiliary polynomial H whose common useful-slot value h_m is positive and satisfies

$$h_m\Delta' \geq \max\{1, rc'_e\}, \qquad |H(\xi)| \leq 1 \text{ at every target unused slot } \xi.$$

This search can be realized by the method from Section 3 with weight $w > 1$. Finally, we multiply both stored RLWE components by H while keeping the decryption factor F'. This changes the encoding scale to $h_m\Delta'$ and preserves the bound $K|F'(\xi)|$ on unused slots, allowing us to use $\rho' = h_m\Delta'$. However, we found by experiments that the minimum c_m of such H grows rapidly with k, in which case H provides coarse bit granularity for useful-slot adjustment. Whether HomPMult or mult-by-H is more suitable therefore depends on the concrete parameters.

Relation to w and w'. Suppose $\rho = \Delta \approx rc_e > 1$ and the ideal estimates $c_e \propto w^{-4/k}$ and $c_m \propto w^{1-4/k}$ hold, then

$$\frac{\rho'_{\min}}{\rho} \approx \left(\frac{w'}{w}\right)^{-4/k}, \qquad \frac{\Delta'}{\Delta} \approx \left(\frac{w'}{w}\right)^{1-4/k}, \qquad \frac{\rho'_{\min}}{\Delta'} \approx \frac{w}{w'},$$

where $\rho'_{\min}$ denotes the smallest divisor required by unused-slot control.

If $w' < w$, the target encoding supports higher precision, but its required rescaling divisor exceeds the converted encoding scale by an estimated factor w/w'. An auxiliary H as above is then used to amplify the useful slots. If $w' > w$, the converted encoding scale exceeds $\rho'_{\min}$, so we may use $\rho' = \Delta'$.

4.3 Changing the Number of Useful Slots

Conversions that change k will change the number of useful slots, which is equal to $2N/k$. Slots that retain their type follow the preceding analysis, so we only consider slots whose type changes.

From unused to useful slots. When $k' < k$, the target has more useful slots. Each newly useful slot contains garbage value with bound

$$|(uF')(\xi)| \leq Kc'_m,$$

whose magnitude is at most Kc_m/Δ when viewed relative to the target encoding scale $\Delta' = \Delta c'_m/c_m$. These garbage entries can be removed by a subsequent plaintext mask.

From useful to unused slots. When $k' > k$, some source useful slots become unused. Their values must be zeroed before conversion, otherwise the converted unused-slot value may exceed the noise bound. Before changing the factor, multiply by an encoded zero–one mask and rescale. The mask retains the target useful slots and is zero at the useful slots to be discarded.

Let ρ_{mask} be the mask encoding scale and the rescaling factor. Choose $\rho_{\mathrm{mask}} \geq \max\{1, rc_e\}$ and $\rho_{\mathrm{mask}} \geq |u(\xi)|$ for every discarded useful slot value $u(\xi)$. At such a slot, the encoded mask consists only of rounding error, bounded by $za_\xi\sqrt{V_{\mathrm{rnd}}}$. The decrypted output therefore has magnitude at most

$$\frac{|u(\xi)|}{\rho_{\mathrm{mask}}} za_\xi \sqrt{V_{\mathrm{rnd}}} + za_\xi \sqrt{V_{\mathrm{ms}}} \leq a_\xi z\big(\sqrt{V_{\mathrm{rnd}}} + \sqrt{V_{\mathrm{ms}}}\big).$$

After replacing F with F', the bound becomes

$$|F'(\xi)|z(\sqrt{V_{\mathrm{rnd}}} + \sqrt{V_{\mathrm{ms}}}) \leq K|F'(\xi)|,$$

additionally requiring $K \geq z(\sqrt{V_{\mathrm{rnd}}} + \sqrt{V_{\mathrm{ms}}})$ for noise control.

For the already-unused slots, the mask preserves the invariant bound K because it is a size-one linear transformation with $\rho_{\mathrm{mask}} \geq \max\{1, rc_e\}$. The retained useful slots also keep their encoding scale through masking.

Finally, the multiplication by a plaintext mask required by scale re-adjustment can be fused with the slot-selection mask, saving one layer.

4.4 Conversion in General Cases

A conversion between arbitrary admissible (k, w) and (k', w') can be realized as one change of k and one change of w. The required mask is applied before useful slots are discarded. We then replace the factor by the final F', determine the final encoding scale and unused-slot requirement, and apply an auxiliary H only if useful-slot amplification is needed.

5 Bootstrapping for GCKKS

Considering the high-level similarity of GCKKS to GBFV, it is natural to adopt a GBFV-like bootstrapping workflow [15]. However, this approach requires three levels in the unreduced plaintext scale Δ/c_m. This modulus consumption greatly limits the available modulus after bootstrapping, and potentially invalidates the bootstrapping in high-precision scenarios.

On the other hand, we find Choe et al.'s high-precision CKKS bootstrapping valuable for GCKKS bootstrapping [11]. It adopts the EvalRound bootstrapping paradigm [19] and discrete CKKS [12,1,2] to handle only the low-precision overflow part, thereby reducing the number of high-precision rescalings to four. To realize GCKKS bootstrapping, these high-precision levels are replaced with small-precision ones using the ciphertext conversion method in the previous section.

5.1 Workflow of GCKKS Bootstrapping.

Given a GCKKS ciphertext of $c = (b, a) \in R_q^2$ with decryption factor F and decryption result $m \in R$, its bootstrapping consists of the following stages.

1. **Convert to standard CKKS.** Absorb F into the RLWE components to produce a standard CKKS ciphertext $(bF, aF) \in R_q^2$. This conversion preserves the decryption result m and thereby the slots. The modulus q is chosen such that $\|m\|_\infty$ is several bits smaller than q.
2. **Modulus raising.** Balance-reduce (bF, aF) modulo q and reinterpret the result modulo $Q \gg q$. The resulting ciphertext has a linear decryption result of $m + qI$. The overflow polynomial I satisfies $\|I\|_\infty \leq 15$ with overwhelming probability, when using the sparse secret encapsulation with a secret Hamming weight of 32 [3].
3. **Low-precision coefficents to slots.** The CtS step moves the N coefficients of I into the slots of two CKKS ciphertexts by homomorphically evaluating iDFT [4,6]. CtS uses a small scaling factor independet of w, because it is only required to preserve I but not the lower bits occupied by m.
4. **Bit extraction.** The two incoming ciphertexts are viewed as discrete CKKS ciphertexts with modulus 32. We then use functional bootstrapping to decompose the coefficients of I into bits.
5. **Iterative ciphertext conversion and precision doubling.** For each ciphertext, we split it into two ciphertexts with doubled k and doubled encoding precision. Then, the error cleaning polynomials from [8] are used to reduce the error in the bits. This step is repeated until the value of k reaches that of the initial ciphertext c. This step produces $\log_2(32) \cdot k/2$ GCKKS ciphertexts.
6. **Recombine the bits.** The bits are recombined into the coefficients of I via a linear combination of ciphertexts. This step produces $k/2$ ciphertexts.
7. **Recovering the useful-slot values of I.** We use homomorphic linear transformation to recover the useful-slot values of qI inside a single ciphertext.
8. **Restoring the original F factor.** We convert the mod-raised ciphertext into a GCKKS ciphertext with decryption factor F. This step preserves the useful-slot values while refreshing the unused-slot values.
9. **Subtraction.** We subtract the output ciphertext of Step 7 from that of Step 8. This recovers the useful-slot values of c and finishes the bootstrapping.

Compared to the bootstrapping workflow in [11], steps 2 to 4 are directly adopted from the original workflow, steps 5, 6, 7, and 9 are modifed to be compatible with GCKKS, while the format conversion steps (1 and 8) are exclusive for GCKKS. In the following, we give more details on steps 5–9, since the correctness of step 1 is trivial and steps 2–4 are identical to those from Choe et al.'s method.

5.2 Iterative Ciphertext Split and Precision Doubling

After the previous steps, we end with the bits of I's coefficients inside the slots of several standard CKKS ciphertexts. This step aims to iteratively double both the

bitlength of the CKKS-equivalent scaling factor and the message precision of the encrypted values. The former is realized by splitting the input ciphertexts into two ciphertexts with a doubled value of k, so that the CKKS-equivalent scaling factor is approximately squared while keeping the GCKKS scaling factor stable. The latter is realized by evaluating the order-two error cleaning polynomial $h_1(x) = 3x^2 - 2x^3$ [8], following the approach by BLEACH [12] and Choe et al. [11].

We note that the design must take into account the useful-slot precision, the unused-slot noise control, and the efficiency of the algorithm. In the following, we first focus on useful-slot computation, and discuss how to patch it with unused-slot noise analysis.

The precision doubling procedure. In this step, we need to split the input ciphertext with a p-bit encoding precision into two ciphertexts with $2p$-bit encoding precision and halved slots. During the split, a direct reinterpretation into the target F' roughly divides the useful-slots by 2^p, producing useful-slot values far below the rescaling factor. To compensate for this, the general conversion methods from Section 4.2 uses `HomPMult` or auxiliary H to increase the useful-slot size, which seems a waste of the encoding capability in this case.

In an attempt to avoid this rapid change of useful-slot magnitude, we observe that the ciphertext after tensoring stage in a `HomCMult` has a decryption factor of F^2, whose useful-slot value is close to that of the target F'. Therefore, the useful-slot magnitudes change more smoothly during the conversion from F^2 to F'. To achieve better efficiency, we also choose to delay the ciphertext split as much as possible, so that a larger portion of the computation only needs to be carried out on a single ciphertext, rather than on two.

The dataflow of the evaluation of h_1 is demonstrated in Figure 1. Although the figure focuses on processing the even-indexed slots, the workflow for odd-indexed slots is similar. Simply left-rotate the useful slots by one position (without key-switching), perform the same operations used to handle even-indexed slots, and key-switch to the original secret before the final rescaling. We also choose F' so that the CKKS-equivalent scale is squared in this procedure, i.e.

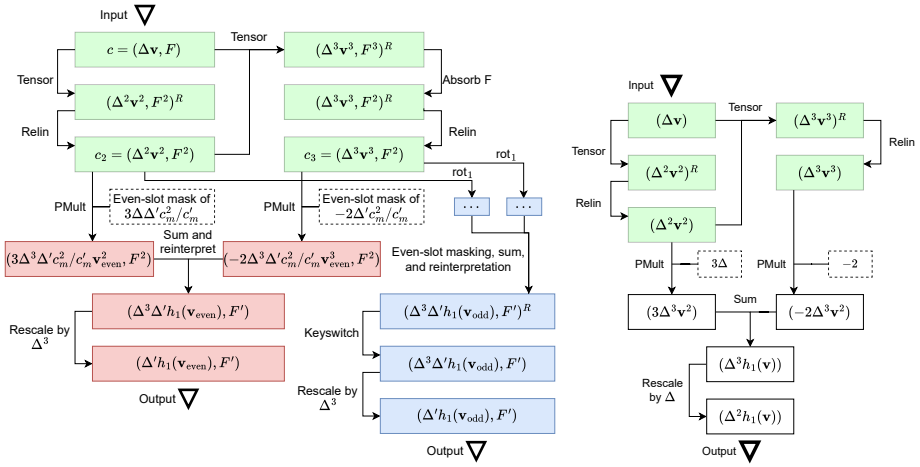
$$\Delta'/c'_m \approx \Delta^2/c_m^2.$$

According to [20, Theorem 2 and Fig. 3], let $u := \log_2(\Delta/c_m)$, then there exists some constant $a > 0$ such that $\log_2(\Delta') \approx (1 - 2/k)a + 4u/k$ and $\log_2(\Delta) \approx (1 - 4/k)a + 4u/k$. Therefore, the bitlength of the scaling factor increases by at most a bits throughout all the iterations.

The ideal noiseless masks $L'_2, L'_3 \in \mathbb{R}[X]/(X^N + 1)$ have values of $3\Delta\Delta'/c'_m$ and $-2\Delta'/c'_m$ at the even-indexed useful slots and zero at other slots. The two plaintext polynomials encodes L'_2 and L'_3 under the decryption factor F^2

$$L_2 = F^2\lfloor L'_2 \rceil, \qquad L_3 = F^2\lfloor L'_3 \rceil.$$

They are multiplied with c_2 and c_3 in the figure, respectively. Their useful slot contain the additional factor c_m^2/c'_m, which is intended to cancel the effect of reinterpretation from F^2 to F' on the surviving useful slots.



(a) Precision doubling in GCKKS. (b) In CKKS [11].

Fig. 1. Precision doubling procedure in GCKKS and CKKS. Each ciphertext is represented by its useful-slot vector and decryption factor. We abuse the notation $\mathbf{v}^n$ for a component-wise n-th power. The superscript R means the ciphertext requires a relinearization or key-switching. The computation of powers of $\mathbf{v}$ are painted in green, while the even-slot and odd-slot branches are painted in red and blue, respectively.

Useful-slot noise analysis. We want the noise introduced by the homomorphic operations to be small enough, so that it does not affect the noise cleaning property of h_1. The only operations that introduce additional noise are the two relinearizations, the multiplication with mask polynomials, the key-switching for the odd-slot branch, and the final rescaling.

Let the CKKS-equivalent scaling factor be $\Delta_{\text{eq}} = \Delta/c_m$. For the relinearization, it has a decryption factor of F^2 and a scaling factor no less than Δ^2, so its equivalent CKKS scaling factor is $\Delta^2/c_m^2 = \Delta_{\text{eq}}^2$. Also, according to our choice of F', the equivalent scales for both rescaling and L_3 are $\Delta'/c_m' \approx \Delta_{\text{eq}}^2$, while that for L_2 is about $\Delta\Delta_{\text{eq}}^2$. Therefore, the operations have a relative precision similar to CKKS operations under a scaling factor of Δ_{eq}^2. Finally, the key-switching noise in the odd-slot branch has limited impact on the resulting noise, since it is decreased by Δ^3 times in the rescaling.

On the level consumption. The design above costs three levels of scaling factor Δ per iteration, whereas the thrifty method in CKKS bootstrapping costs only one level of Δ, as demonstrated in the right subfigure of Figure 1. The first extra level comes from the output scale. The CKKS version ends with a scaling factor of Δ^2, while the GCKKS version needs to output under a scaling factor of Δ' that is only slightly large than Δ. The second layer is from the additional multiplication with the masking polynomial.

Unused-slot noise control. Unused slots in the output ciphertexts either remain unused from the input ciphertext or come from discarded useful slots. Let d, d_2, and d_3 be the decryption polynomials of c, c_2, and c_3 under F, F^2, and F^2, respectively. With relinearization errors e_1, e_2 on the RLWE components, they satisfy

$$d_2 = d^2 + F^2 e_1, \qquad d_3 = dd_2 + F^2 e_2.$$

Write $r_i = \lfloor L_i' \rceil - L_i'$ for $i \in \{2, 3\}$, and let e_{ms} be the final rescaling error on the RLWE components. Use the bounds B_{rnd}, B_{ks}, and B_{ms} for r_i, e_j, and e_{ms}, respectively.

Slots that remain unused. At an unused embedding point ξ, let $a_\xi = |F(\xi)|$ and suppose $|d(\xi)| \le a_\xi K$. The two branch bounds are

$$|d_2(\xi)| \le a_\xi^2(K^2 + B_{\mathrm{ks}}), \qquad |d_3(\xi)| \le a_\xi^3(K^3 + KB_{\mathrm{ks}}) + a_\xi^2 B_{\mathrm{ks}}.$$

Since $L_2'(\xi) = L_3'(\xi) = 0$, the encoded masks satisfy $L_i(\xi) = F(\xi)^2 r_i(\xi)$. After conversion to F' and rescaling by Δ^3, the output satisfies

$$d_{\mathrm{out}}(\xi) = F'(\xi)\frac{d_2(\xi)r_2(\xi) + d_3(\xi)r_3(\xi)}{\Delta^3} + F'(\xi)e_{\mathrm{ms}}(\xi).$$

Thus the normalized unused-slot bound is

$$\frac{|d_{\mathrm{out}}(\xi)|}{|F'(\xi)|} \le B_{\mathrm{ms}} + \frac{B_{\mathrm{rnd}} a_\xi^2}{\Delta^3}\big(K^2 + 2B_{\mathrm{ks}} + a_\xi(K^3 + KB_{\mathrm{ks}})\big). \tag{2}$$

For $\Delta \ge rc_e$, the second term is at most

$$B_{\mathrm{rnd}}\left(\frac{K^2 + 2B_{\mathrm{ks}}}{r^2\Delta} + \frac{K^3 + KB_{\mathrm{ks}}}{r^3}\right).$$

Since $K < r$ and a standard choice of r satisfies $r > 4B_{\mathrm{ks}}$, the expression above is dominated by B_{rnd}. Then (2) imposes an additional requirement of $K \ge B_{\mathrm{ms}} + B_{\mathrm{rnd}}$ on the normailzed bound r, which coincides with the requirement by useful-to-unused slot conversion in Section 4.3.

Discarded useful slots. At a discarded useful embedding point η, the encoded masks have magnitude at most $c_m^2 B_{\mathrm{rnd}}$. The factor c_m^2 is cancelled during the conversion from F^2 to F', giving

$$\frac{|d_{\mathrm{out}}(\eta)|}{|F'(\eta)|} \le B_{\mathrm{ms}} + \frac{B_{\mathrm{rnd}}}{\Delta^3}\big(|d_2(\eta)| + |d_3(\eta)|\big).$$

Suppose $|d(\eta)| \le \Delta(1+\varepsilon)$ with $\varepsilon \ge 0$, the right-hand side is at most

$$B_{\mathrm{ms}} + B_{\mathrm{rnd}}\left((1+\varepsilon)^3 + \frac{(1+\varepsilon)^2}{\Delta} + \frac{c_m^2 B_{\mathrm{ks}}}{\Delta^3}\big(\Delta(1+\varepsilon) + 2\big)\right).$$

Similar to the previous case, the newly unused slots still have a normalized bound close to $B_{\mathrm{rnd}} + B_{\mathrm{ms}}$ since ε is small.

5.3 Useful-slot Recovery of I

After the end of step 6, we end up with $k/2$ GCKKS ciphertexts storing the coefficiens of I in high precision. To recover the useful-slot values of I is equivalent to evaluating a multi-ciphertext linear transformation over all these ciphertexts. However, the coefficents of I has a highly-structured layout inside the slots, so that this linear transformation has a simple evaluation.

An intuitive way to understand this is by looking at the output ciphertext of step 3, whose slot values are essentially low-precision versions of those outputted by step 6 (up to some permutation). On this ciphertext, to recover the useful-slot value of I is equivalent to evaluate a standard StC and keep only the useful-slot outputs. Following the standard StC decomposition [4,6], the i-th FFT matrix in StC requires homomorphic rotation by $\pm 2^i$ (with i starting from zero).

Specifically, after applying the first StC matrix, the output even-indexed slots become a linear combination of the even- and odd-indexed slots from the input ciphertext. Recall that the first iteration of the precision-doubling stage also splits the even- and odd-indexed slots into two ciphertexts. Therefore, if the precision-doubling stage stops after the first iteration, the first FFT matrix in the multi-ciphertext StC of step 7 can be simply evaluated with a linear combination for each sized-four group of ciphertexts[3], without homomorphic rotations.

This property actually applies recursively to all the $\log_2(k/4)$ iterations in the precision-doubling stage. Then, the first $\log_2(k/4)$ FFT matrices in StC can be implemented via a linear combination of all $k/2$ ciphertexts, producing a single ciphertext. An example of the first two layers are given in Figure 2. The remaining FFT matrices involve homomorphic rotation with offset divisible by $k/4$, which are valid GCKKS rotations. Therefore, these matrices can be evaluated as a single-ciphertext partial StC using standard techniques on the ciphertext.

Unused-slot noise control. We view step 6 and step 7 as a whole linear transformation. The unused-slot noise analysis in the linear combination of $\log_2(32) \cdot k/2$ ciphertexts is compatible with existing noise control model. Alternatively, if the linear-combination has a size $\log_2(32) \cdot k/2$ exceeding the limit of noise control conditions, it can be factorized into multiple smaller linear combinations using the natural factorization of StC.

For the final single-ciphertext $2N/k$-sized linear transformation, it is usually evaluated via BSGS, which is not directly compatible with the noise model. For a size-L transformation with giant step $g \mid L$, the BSGS algorithm homomorphically evaluates the linear transformation $\mathbf{T}$ as

$$\mathrm{Enc}(\mathbf{Av}) = \sum_{i=0}^{L/g-1} \mathrm{HomRot}_{gi}\big(\sum_{j=0}^{g-1} \kappa_{i,j} \cdot \mathrm{HomRot}_j(\mathrm{Enc}(\mathbf{v}))\big),$$

[3] The ciphertext number is four instead of two because FFT mixes complex values, whereas the ciphertexts after step 6 store real values.

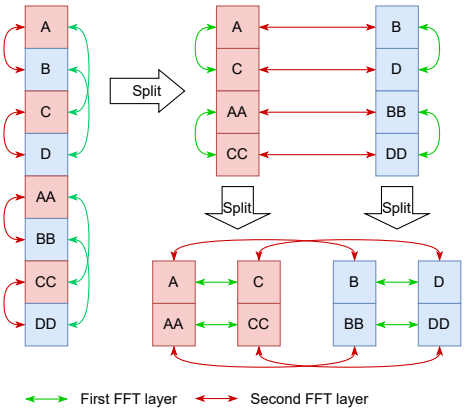


Fig. 2. The slot-mixing behavior of the first two FFT levels in StC. Mixed slots are connected with arrows. The 'A,B,C,D' are names of the slot positions and do not represent slot values.

where $\kappa_{i,j}$ are plaintext polynomials encoding the diagonals of $\mathbf{A}$. We use lazy rescaling to control the key-switching noise in the outer loop.

At an unused embedding point ξ, each $\kappa_{i,j}$ has its unused-slot bounded by $a_\xi B_{\mathrm{rnd}}$, while the key-switching errors are bouned by $a_\xi B_{\mathrm{ks}}$. Recall that the slots connected by valid GCKKS rotations have the same bound B_ξ. Therefore, at an unused embedding point ξ, the inner loop ciphertext has a unused-slot bound of

$$g a_\xi B_{\mathrm{rnd}}(a_\xi B_{\mathrm{ks}} + B_\xi),$$

while that for the output ciphertext is

$$1/\rho(L a_\xi B_{\mathrm{rnd}}(a_\xi B_{\mathrm{ks}} + B_\xi) + L/g a_\xi B_{\mathrm{ks}}) + a_\xi B_{\mathrm{ms}}.$$

For $\rho \geq 1$ and $\rho/a_\xi \geq r$, dividing this bound by a_ξ gives the normalized propagation upper bound

$$g_{\mathrm{BSGS},r}(B) := \frac{L B_{\mathrm{rnd}}}{r}(B + B_{\mathrm{ks}}) + \frac{L}{g} B_{\mathrm{ks}} + B_{\mathrm{ms}},$$

where $B = B_\xi/a_\xi$. When $r > L B_{\mathrm{rnd}}$, this affine map has fixed point

$$B_{\mathrm{BSGS}}(r) = \frac{L B_{\mathrm{rnd}} B_{\mathrm{ks}}/r + (L/g) B_{\mathrm{ks}} + B_{\mathrm{ms}}}{1 - L B_{\mathrm{rnd}}/r}.$$

In addition to existing noise control conditions, we require $B_{\mathrm{BSGS}}(r) \leq K < B^+_{\mathrm{mult}}(r)$. Using $B_{\mathrm{rnd}} = z\sqrt{V_{\mathrm{rnd}}}$, $B_{\mathrm{ks}} = z\sqrt{V_{\mathrm{ks}}}$, and $B_{\mathrm{ms}} = z\sqrt{V_{\mathrm{ms}}}$, a possible choice of r satisfying this additional requirement is

$$r = \max\big(2L B_{\mathrm{rnd}}, (2 + 2L/g) B_{\mathrm{ks}} + 4 B_{\mathrm{ms}}\big).$$

5.4 Useful-slot Preserving Conversion from CKKS to GCKKS

Step 9 needs to convert a CKKS ciphertext into a GCKKS ciphertext while preserving its useful-slot values. This would be easy if F were a unit, since the GCKKS ciphertext (bF^{-1}, aF^{-1}) decrypts to the same value of $b + as = qI + m$ under a decryption factor of F. However, this becomes nontrivial because we have chosen not to use unit F due to their poorer encoding quality in very-high precision. Another problem is that such conversion with a unit F leaves $O(q)$-sized values in the unused slots. Decreasing the magnitude of these slots may require an extra rescaling by a factor as large as q.

To solve these issues, our idea is to re-encode the RLWE components, which preserves the decrypted useful-slot values while resetting the unused slots with fresh rescaling noises. The following map is applied independently to b and a

$$\begin{array}{ccccccccc} \mathbb{Z}[X]/(X^N+1) & \xrightarrow{\substack{\text{reduce}\\ \text{and scale}}} & \mathbb{C}[X]/(X^{2N/k}-\zeta_k) & \xrightarrow{\texttt{Lift}_{N,k}} & \mathbb{R}[X]/(X^N+1) & \xrightarrow{\text{round}} & \mathbb{Z}[X]/(X^N+1) & \xrightarrow{\times F} & \mathbb{Z}[X]/(X^N+1) \\ U & \mapsto & u' = c_m^{-1}(U \bmod X^{2N/k}-\zeta_k) & \mapsto & U' = \texttt{Lift}_{N,k}(u') & \mapsto & U^* = \lfloor U' \rceil & \mapsto & U^* \cdot F \end{array}$$

Note that the workflow is almost identical to the encoding procedure of F-encoder, except for the first step. The final multiplication by F is realized by setting the decryption factor to F, without changing the stored ring element. The full algorithm is presented in Algorithm 1, and its correctness is proven in Lemma 3.

Algorithm 1 Useful-slot Preserving Conversion from CKKS to GCKKS

Require: A CKKS ciphertext $c = (b, a) \in R_Q^2$, target parameter k, and decryption factor F with common useful-slot value c_m.

Ensure: A GCKKS ciphertext $c^* = (b^*, a^*) \in R_Q^2$ with decryption factor F, having similar useful-slot values as c.

1: **for** each component $U \in \{b, a\}$ **do**
2: $u' \leftarrow U \bmod (X^{2N/k} - \zeta_k)$
3: $u' \leftarrow c_m^{-1} a'$
4: $U' \leftarrow \texttt{Lift}_{N,k}(u')$ ▷ CRT reconstruction with zero unused slots
5: $U^* \leftarrow \lfloor U' \rceil$ ▷ Denote the obtained values by b^* and a^*
6: **end for**
7: **return** $c^* = (b^* \bmod Q, a^* \bmod Q)$ with decryption factor F

Lemma 3. *Using the notation of Algorithm 1, let B' and A' be the unrounded CRT lifts of b and a, respectively. Let $e = (\lfloor B' \rceil - B') + (\lfloor A' \rceil - A')s$, suppose that*

$$\|b + as\|_2 + \|eF\|_\infty < Q/2.$$

Then the output ciphertext c^* satisfies

$$\begin{aligned} \tau_{useful}(\texttt{Dec}_{s,F}(c^*)) - \tau_{useful}(b + as) &= c_m \tau_{useful}(e), \\ \tau_{unused}(\texttt{Dec}_{s,F}(c^*)) &= \tau_{unused}(eF). \end{aligned}$$

Proof. We first consider the integer components $b^* = \lfloor B' \rceil$ and $a^* = \lfloor A' \rceil$ before reduction modulo Q. At each useful embedding point ξ, the reduction and scaling steps give

$$B'(\xi) = c_m^{-1} b(\xi), \qquad A'(\xi) = c_m^{-1} a(\xi).$$

Since $F(\xi) = c_m$, the polynomial $(B' + A's)F$ agrees with $b + as$ at every useful embedding point. At every unused embedding point, both B' and A' have zero values, so $(B' + A's)F$ is also zero at this slot. The definition of e gives

$$(b^* + a^* s)F = (B' + A's)F + eF.$$

Applying the useful and unused embeddings proves the claimed identities before reduction modulo Q.

It remains to show that centered decryption recovers this unreduced polynomial. Since $(B' + A's)F$ shares the same useful-slot values as $b + as$ and has zero unused slots, we have $\|(B' + A's)F\|_2 \leq \|b + as\|_2$ due to canonical embedding being a scaled isometry. Consequently,

$$\|(b^* + a^* s)F\|_\infty \leq \|((B' + A's)F)\|_\infty + \|(eF)\|_\infty \leq \|b + as\|_2 + \|eF\|_\infty < Q/2.$$

Thus $(b^* + a^* s)F$ is the unique centered representative of its residue modulo Q. Reducing the components of c^* modulo Q does not change this residue, so $\mathtt{Dec}_{s,F}(c^*) = (b^* + a^* s)F$, and both identities follow. □

5.5 Complexity and Modulus-consumption Analysis

Let p denote the target encoding precision in bits. We count homomorphic ring operations separately from their cost at the working ciphertext modulus. Modulus consumption is measured along a ciphertext path, since parallel branches use the same modulus chain.

Precision doubling. Starting from $k = 4$, this stage performs $\log_2(k/4)$ splitting iterations. The number of ciphertexts doubles at every iteration and ends at $\log_2(32) \cdot k/2$. Since each intermediate ciphertext requires three key-switchings, as shown in Figure 1, the total key-switching count is bounded by $15k$.

Each iteration consumes three layers, and each layer has a scaling factor bitlength of $O(p/k)$, giving a modulus consumption of $O((p/k)\log k)$ bits.

Remaining steps. Bit recombination and the part of StC acting across ciphertexts require $O(k)$ plaintext multiplications and additions for the fixed five-bit decomposition. The remaining $2N/k$-slot transformation can be evaluated by standard BSGS and level collapsing techniques, where each collapsed linear transformation consumes $O(p/k)$ bits.

All other steps have a precision unrelated to p, so the dependence of the overall consumed modulus on p is determined by the precision-doubling and recovery stages above. In total, this gives a GCKKS bootstrapping of $O(k)$ homomorphic operations within a modulus consumption of $O(p \log k/k)$ bits.

6 Implementation

Not available yet.

References

1. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits with CKKS. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, Part II. Lecture Notes in Computer Science, vol. 14652, pp. 94–123. Springer, Cham, Switzerland, Zurich, Switzerland (May 26–30, 2024). https://doi.org/10.1007/978-3-031-58723-8_4
2. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with CKKS. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024, Part I. Lecture Notes in Computer Science, vol. 15484, pp. 330–360. Springer, Singapore, Singapore, Kolkata, India (Dec 9–13, 2024). https://doi.org/10.1007/978-981-96-0875-1_11
3. Bossuat, J.P., Troncoso-Pastoriza, J.R., Hubaux, J.P.: Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In: Ateniese, G., Venturi, D. (eds.) ACNS 2022: 20th International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science, vol. 13269, pp. 521–541. Springer, Cham, Switzerland, Rome, Italy (Jun 20–23, 2022). https://doi.org/10.1007/978-3-031-09234-3_26
4. Chen, H., Chillotti, I., Song, Y.: Improved bootstrapping for approximate homomorphic encryption. In: Ishai, Y., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2019, Part II. Lecture Notes in Computer Science, vol. 11477, pp. 34–54. Springer, Cham, Switzerland, Darmstadt, Germany (May 19–23, 2019). https://doi.org/10.1007/978-3-030-17656-3_2
5. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) Advances in Cryptology – ASIACRYPT 2011. Lecture Notes in Computer Science, vol. 7073, pp. 1–20. Springer Berlin Heidelberg, Germany, Seoul, South Korea (Dec 4–8, 2011). https://doi.org/10.1007/978-3-642-25385-0_1
6. Cheon, J.H., Han, K., Hhan, M.: Faster homomorphic discrete Fourier transforms and improved FHE bootstrapping. Cryptology ePrint Archive, Report 2018/1073 (2018), https://eprint.iacr.org/2018/1073
7. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017, Part I. Lecture Notes in Computer Science, vol. 10624, pp. 409–437. Springer, Cham, Switzerland, Hong Kong, China (Dec 3–7, 2017). https://doi.org/10.1007/978-3-319-70694-8_15
8. Cheon, J.H., Kim, D., Kim, D.: Efficient homomorphic comparison methods with optimal complexity. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2020, Part II. Lecture Notes in Computer Science, vol. 12492, pp. 221–256. Springer, Cham, Switzerland, Daejeon, South Korea (Dec 7–11, 2020). https://doi.org/10.1007/978-3-030-64834-3_8
9. Cheon, J.H., Lee, J.: High-precision CKKS with fixed modulus consumption. Cryptology ePrint Archive, Paper 2026/2531 (2026), https://eprint.iacr.org/archive/2026/2531/20261018:191446, initial version, 18 October 2026
10. Cheon, J.H., Lee, J.: High-precision CKKS with focal units. Cryptology ePrint Archive, Paper 2026/2531 (2026), https://eprint.iacr.org/archive/2026/2531/20261020:155137, revision of 20 October 2026

11. Choe, H., Kim, J., Stehlé, D., Suvanto, E.: Leveraging discrete CKKS to bootstrap in high precision. In: Huang, C.Y., Chen, J.C., Shieh, S.P., Lie, D., Cortier, V. (eds.) ACM CCS 2025: 32nd Conference on Computer and Communications Security. pp. 1083–1097. ACM Press, Taipei, Taiwan (Oct 13–17, 2025). https://doi.org/10.1145/3719027.3765080
12. Drucker, N., Moshkowich, G., Pelleg, T., Shaul, H.: BLEACH: Cleaning errors in discrete computations over CKKS. Journal of Cryptology **37**(1), 3 (Jan 2024). https://doi.org/10.1007/s00145-023-09483-1
13. Dumezy, J.: Fully homomorphic encryption for small batches of complex numbers. Cryptology ePrint Archive, Paper 2026/2513 (2026), https://eprint.iacr.org/2026/2513
14. Gama, N., Nguyen, P.Q.: Predicting lattice reduction. In: Smart, N.P. (ed.) Advances in Cryptology – EUROCRYPT 2008. Lecture Notes in Computer Science, vol. 4965, pp. 31–51. Springer Berlin Heidelberg, Germany, Istanbul, Turkey (Apr 13–17, 2008). https://doi.org/10.1007/978-3-540-78967-3_3
15. Geelen, R., Vercauteren, F.: Better GBFV bootstrapping and faster encrypted edit distance computation. IACR Communications in Cryptology (CiC) **2**(4), 35 (2026). https://doi.org/10.62056/a3n56chdj
16. Halevi, S., Shoup, V.: Algorithms in HElib. In: Garay, J.A., Gennaro, R. (eds.) Advances in Cryptology – CRYPTO 2014, Part I. Lecture Notes in Computer Science, vol. 8616, pp. 554–571. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 17–21, 2014). https://doi.org/10.1007/978-3-662-44371-2_31
17. Halevi, S., Shoup, V.: Faster homomorphic linear transformations in HElib. In: Shacham, H., Boldyreva, A. (eds.) Advances in Cryptology – CRYPTO 2018, Part I. Lecture Notes in Computer Science, vol. 10991, pp. 93–120. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 19–23, 2018). https://doi.org/10.1007/978-3-319-96884-1_4
18. Kim, D., Song, Y.: Approximate homomorphic encryption over the conjugate-invariant ring. In: Lee, K. (ed.) ICISC 18: 21st International Conference on Information Security and Cryptology. Lecture Notes in Computer Science, vol. 11396, pp. 85–102. Springer, Cham, Switzerland, Seoul, Korea (Nov 28–30, 2019). https://doi.org/10.1007/978-3-030-12146-4_6
19. Kim, S., Park, M., Kim, J., Kim, T., Min, C.: EvalRound algorithm in CKKS bootstrapping. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022, Part II. Lecture Notes in Computer Science, vol. 13792, pp. 161–187. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). https://doi.org/10.1007/978-3-031-22966-4_6
20. Ma, S., Liu, J., Wang, A., Wang, X.: Homomorphic encryption for high-precision arithmetic of approximate numbers. Cryptology ePrint Archive, Paper 2026/2524 (2026), https://eprint.iacr.org/2026/2524
21. Ma, S., Liu, J., Wang, A., Wang, X.: Homomorphic encryption for high-precision arithmetic of approximate numbers. Cryptology ePrint Archive, Paper 2026/2524 (2026), https://eprint.iacr.org/archive/2026/2524/20261018:022506, initial version, 18 October 2026